

平台仓一致性: 完整端到端示例

以 `NOAETCSwitchSt`（NOA ET关控制状态）为一个 SOME/IP 信号为例，从 JSON 配置一直到运行时数据流动，每层标出代码在哪里。

第一步: JSON 配置 → 生成工具

输入 `x_dom_someip_rt_fawhq_e001_l0.json` 里定义了这个服务:

```
// JSON 伪代码
route "dji_application" → consumed_events: [{
  service: "HMI_Dis_NOAControl",
  event:   "Dis_NotifyONChgNOAETCSwitchSt",
  struct:  "cpp_HMI_ETCSwitchCmd_enum",
  cycle_ms: 100
}]
```

生成工具(v0.17) 读取 JSON + DBC/ARXML, 输出 7 个文件到 `x_dom_someip_rt_gen/`:

生成文件	内容	车型相关?
<code>x_dom_someip_rt_cfg.h</code>	Service ID 宏 + extern 对象声明	每车型不同
<code>x_dom_someip_rt_cfg.cpp</code>	SvcItemInfo 实例化 + map 注册 + <code>x_dom_someip_rt_prod_cfg()</code>	每车型不同
<code>x_dom_someip_rt_protocol.hpp</code>	所有 cpp_ 结构体 + Serialize/Deserialize 函数	每车型不同
<code>x_dom_someip_rt_sigif.h</code>	SigIf_Get/Set/Callback API 声明	每车型不同
<code>x_dom_someip_rt_sigif_get.cpp</code>	SigIf_Get 实现 + <code>SigIf_OnRecvData()</code> 分发	每车型不同
<code>x_dom_someip_rt_sigif_set.cpp</code>	SigIf_Set 实现	每车型不同
<code>x_dom_someip_rt_dbg.cpp</code>	shell debug dump	每车型不同

第二步: 生成代码给平台仓 “注册” 自己

生成的 `x_dom_someip_rt_cfg.cpp` 提供了 `x_dom_someip_rt_prod_cfg()` 函数:

```
// ===== 生成代码: cfg.cpp =====

// ① 为每条SOME/IP event 实例化一个 SvcItemInfo (平台仓类型)
x_dom_someip_rt::SvcItemInfo
someip_recv_...NOAETCSwitchSt{
  16,                                     // Service ID   → 生成
  "HMI_Dis_NOAControl_Dis_NotifyONChgNOAETCSwitchSt", // 名称         → 生成
  100,                                     // 周期100ms    → 生成
  SigIf_OnRecvData, // - 回调入口 (生成代码自己提供)
  0};                                     // 不立即发送

// ② 注册到全局map (key=Service ID, value=SvcItemInfo指针)
static const std::map<uint32_t, x_dom_someip_rt::SvcItemInfo*>
svc_route_recv_svc_info_map = {
  ...
  {16, &someip_recv_...NOAETCSwitchSt}, // ID 16 → 该对象
  ...
};

// ③ 注册函数: 将车型特定的map指针注入平台仓全局配置
extern "C" int32_t x_dom_someip_rt_prod_cfg(void)
{
  x_dom_someip_rt_set("app_service_en", (void*)true);
  x_dom_someip_rt_set("topic_node_name", (void*)"ad_app_comif");
  x_dom_someip_rt_set("svc_route_recv_svc_info_map", // - key匹配
    (void*)&svc_route_recv_svc_info_map); // - 车型map
  x_dom_someip_rt_set("read_svc_list", (void*)(read_svc_list));
  // ...
}
```

第三步: 平台仓 `x_dom_someip_rt_set()` 接受注册

平台仓 `x_dom_someip_rt_com_cfg.cpp` — 所有车型同一份代码:

```
// 平台仓全局配置结构体 (所有车型都用这个)
x_dom_someip_config_t g_x_dom_someip_config{}; // 初始化为空

void x_dom_someip_rt_set(const std::string& key, void* value1, ...)
{
  // key-value 匹配: 不管车型, 只管字符串key
  if (key == "svc_route_recv_svc_info_map" && value1 != nullptr)
    g_x_dom_someip_config.svc_route_recv_svc_info_map_ptr =
      = static_cast<std::map<uint32_t, SvcItemInfo*>*>(value1);
  //      ↑ 存的是 void* 指针, 不关心具体是什么车型的map
  //      ↑ 平台仓只知道: "有个 map<uint32_t, SvcItemInfo*> 给我用"
}
```

第四步: 平台仓运行时引擎使用注册的数据

平台仓 `service_route.cpp` — 所有车型同一份代码:

```
// 收到SOME/IP数据时的处理 (由底层someip协议栈触发)
void svc_route_rx_handle(uint32_t id, const uint8_t *data, size_t len)
{
  // 从注册表查找对应的 SvcItemInfo
  auto it = g_x_dom_someip_config
    .svc_route_recv_svc_info_map_ptr->find(id);
  //      ↑ 这个map指针是生成的 cfg.cpp 在 prod_cfg() 里注册的
  if (it != end) {
    SvcItemInfo* item = it->second;
    item->set_origin_data(id, data, len); // 存入原始字节

    if (item->is_immediate_svc())
      svc_route_immediately_pub(id, data, len, item->get_recv_count());
  }
}
```

第五步: 数据到达 → 触发回调链

SvcItemInfo::set_origin_data()) 内部调用了构造函数传入的 (recv_func) (即 SigIf_OnRecvData) :

```
// 平台仓 SvcItemInfo::set_origin_data() 内部 (x_dom_someip_rt_com.h:304-315)
void set_origin_data(uint32_t id, const uint8_t *data, size_t len) {
    m_origin_data = data; // 存原始字节
    if (m_recv_func) // - 就是 SigIf_OnRecvData
        m_recv_func(id); // - 调用生成代码的分发函数
}

// ===== 生成代码: sigif_get.cpp =====
void SigIf_OnRecvData(uint32_t id)
{
    switch (id) {
        case 16: // X_DOM_SOMEIP_RT_SERVER_ID...NOAETCSwitchSt_RX - 生成宏
            if (SigIf_...NOAETCSwitchSt_Cb != nullptr) {
                cpp_HMI_ETCSwitchCmd_enum_data;
                SigIf_...NOAETCSwitchSt_Get(&data);
                //
                SigIf_...NOAETCSwitchSt_Cb(&data, ctx); // 调用用户注册的回调
            }
            break;
            // ... 70+ 个 case
    }
}
```

```
// ===== 生成代码: sigif_get.cpp 中 SigIf_Get 实现 =====
int32_t SigIf_...NOAETCSwitchSt_Get(cpp_HMI_ETCSwitchCmd_enum *pData)
{
    return someip_recv_...NOAETCSwitchSt // - 全局 SvcItemInfo 对象
        .get_usr_data_ext<cpp_HMI_ETCSwitchCmd_enum>(&
            cpp_HMI_ETCSwitchCmd_enum_DataOffset, // - 生成宏
            pData);
    cpp_HMI_ETCSwitchCmd_enum_Deserialize); // - 生成的反序列化函数
}
```

第六步: 应用层使用信号

```
// 用户代码 (适配, 如某个 adapter 或 domain_function 中)
static void on_NOA_ETC_switch_change(const cpp_HMI_ETCSwitchCmd_enum *data, void *ctx) {
    if (data->value == 0x01) { /* ETC ON */ }
    if (data->value == 0x02) { /* ETC OFF */ }
}

void my_init() {
    SigIf_...NOAETCSwitchSt_SetCallback(on_NOA_ETC_switch_change, nullptr);
}
```

平台仓为什么对所有车型都一样

平台仓 (一份代码)
x_dom_someip_rt_com.h
└ class SvcItemInfo - 通用数据容器
└ set_origin_data() - 存原始字节
└ get_usr_data_ext<T>() - 模板反序列化
└ dump_usr_data_ext<T>() - 模板dump
└ 对 T 的类型一无所知!
x_dom_someip_rt_com_cfg.cpp/h
└ g_x_dom_someip_config - 通用配置槽位
└ x_dom_someip_rt_set() - void* 注册接口
service_route.cpp
└ svc_route_rx_handle() - 查map-调set_origin_data
└ svc_route_main_function() - 周期性转发调度
x_dom_someip_rt.cpp
└ x_dom_someip_rt_init() - Topic注册+周期任务
└ x_dom_someip_rt_main_func() - 50ms入口
编译时 include + 链接时符号绑定
运行时 void* 指针注入
生成代码 (每车型不同)
x_dom_someip_rt_cfg.cpp
└ 75个 SvcItemInfo 全局对象
└ svc_route_recv_svc_info_map (map<id, SvcItemInfo>)
└ x_dom_someip_rt_prod_cfg() 注册函数
x_dom_someip_rt_protocol.hpp
└ 75个 cpp_xxx 结构体定义
└ 75对 Serialize/Deserialize 函数
x_dom_someip_rt_sigif_get.cpp
└ SigIf_OnRecvData() - switch(id) 分发
└ 75个 SigIf_xxx_Get() 实现

关键: 平台仓的 SvcItemInfo 类用 template<typename T_CPP, typename DeserializeFunc> 实现 get_usr_data_ext 和 dump_usr_data_ext. 它对 T_CPP 的唯一要求是: (1) 有 nlohmann::json 的序列化支持 (用于 dump), (2) DeserializeFunc 能把它从字节流反序列化出来. 无论 T_CPP 是 cpp_HMI_ETCSwitchCmd_enum 还是 cpp_Sf_PDC_USS_struct 还是 cpp_BO_BLERPAINfo_Struct, 平台仓代码逻辑完全不变——它只是一个带锁的字节容器 + 泛型序列化代理。

总结

层	谁提供	作用
SvcItemInfo 类	平台仓	通用容器: 存原始字节、加锁、计数、超时检测、模板序列化
g_x_dom_someip_config	平台仓	全局配置槽: void* 指针指向车型特定的 map
x_dom_someip_rt_set()	平台仓	注册接口: key-value 字符串匹配, 不依赖车型类型
svc_route_rx_handle()	平台仓	运行时引擎: 查 map - 调 set_origin_data - 触发回调

SvcItemInfo 实例化	生成代码	75 个全局对象，各有不同的 ID、名称、周期
svc_route_recv_svc_info_map	生成代码	车型信号表: 哪些 Service ID 对应哪些 SvcItemInfo
cpp_HMI_ETCSwitchCmd_enum	生成代码	车型数据类型: C++ 结构体定义
SigIf_OnRecvData()	生成代码	分发回调: switch(id) 匹配 75 个 case
x_dom_someip_rt_prod_cfg()	生成代码	注册入口: 调用 x_dom_someip_rt_set() 注入配置

这就是 "模板引擎 + 数据配置分离" 的本质：平台仓是一个用 `void*` 和 `template` 构建的通用框架，生成代码是给它填车型特定的数据、类型和回调函数指针。换一个车型，只需要换掉 include 路径下的 `x_dom_someip_rt_cfg.cpp` 和 `x_dom_someip_rt_protocol.hpp`，平台仓代码完全不动。